

ĐẠI HỌC QUỐC GIA – HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

-----□□□□-----



BÁO CÁO ĐỒ ÁN 2

Xây dựng hệ thống Continuous Deployment (CD) cho YAS – Yet Another Shop

Môn học: CSC11007- Nhập môn DevOps

Lớp: CQ2023/4

GVHD: Nguyễn Thanh Quân

STT	Họ tên	MSSV
1	Nguyễn Lê Hoàng Trung	23122004
2	Lê Hoàng Minh Huy	23122033
3	Châu Văn Minh Khoa	23122035
4	Trần Tạ Quang Minh	23122042

Thành phố Hồ Chí Minh - 2026

I. Mở đầu	3
II. Giải pháp và Triển khai	3
2.1. Tổng quan kiến trúc và môi trường	3
2.2. CI: Build image theo commit và push lên registry	7
2.3. Xây dựng cluster K8S và triển khai YAS	9
2.4. Job developer_build và truy cập theo domain	12
2.5. Job xoá triển khai	14
2.6. Nâng cao: Quản lý dev/staging bằng ArgoCD – GitOps	15
2.7. Nâng cao: Service Mesh với Istio và Kiali	20
2.8. Observability: thu thập log tập trung với Loki, Promtail và Grafana	24
III. Phụ lục	28
3.1. Thông tin nộp bài	28
3.2. Quy trình thiết lập (tóm tắt)	28

I. Mở đầu

Đồ án 2 tập trung xây dựng hệ thống Continuous Deployment (CD) cho YAS (Yet Another Shop), một ứng dụng thương mại điện tử theo kiến trúc microservices. Trên cơ sở quy trình Continuous Integration (CI) đã được xây dựng ở đồ án trước, báo cáo này trình bày cách nhóm tổ chức pipeline tự động từ mã nguồn, container image, cấu hình triển khai, đến vận hành ứng dụng trên Kubernetes.

Thay vì sử dụng Jenkins và Minikube như hướng triển khai gợi ý, nhóm lựa chọn GitHub Actions cho CI/CD và DigitalOcean Kubernetes (DOKS) cho môi trường triển khai. Ứng dụng được đóng gói thành container image và lưu trữ trên GitHub Container Registry (GHCR) dưới namespace ghcr.io/23122004.

Phạm vi triển khai bám theo các yêu cầu chính của đề bài: xây dựng CI để build image theo commit SHA cho từng branch, tạo workflow developer_build cho phép triển khai nhánh của từng service vào môi trường developer và tự động triển khai môi trường dev/staging theo mô hình GitOps với ArgoCD. Ngoài ra, nhóm cấu hình Service Mesh bằng Istio/Kiali để minh họa mTLS, chính sách phân quyền service-to-service và retry policy.

II. Giải pháp và Triển khai

Phần này trình bày chi tiết cách nhóm hiện thực từng yêu cầu, kèm bằng chứng được chụp trực tiếp từ cụm DOKS, GitHub Actions và GHCR đang chạy.

2.1. Tổng quan kiến trúc và môi trường

Hệ thống được triển khai trên DigitalOcean Kubernetes (DOKS) tại region Singapore (sgp1). Cụm sử dụng Kubernetes phiên bản 1.36.0-do.1, gồm 4 worker node loại s-4vcpu-8gb; control plane và worker node do DigitalOcean quản lý. Mô hình này đáp ứng yêu cầu của đề bài về việc xây dựng một môi trường Kubernetes thực thi được ứng dụng YAS, đồng thời giảm phần vận hành thủ công của control plane so với cụm tự dựng.

- Luồng truy cập bên ngoài đi qua NGINX Ingress Controller được expose bằng Service loại LoadBalancer. Ingress định tuyến request dựa trên host name để tách các endpoint như storefront, backoffice, swagger, ArgoCD, Kiali và Grafana.
- Registry image: nhóm sử dụng GitHub Container Registry (GHCR), với quy ước tên image ghcr.io/23122004/yas-<service>. GHCR đóng vai trò container registry trung tâm cho cả CI, developer deployment, dev và staging.
- Hạ tầng dùng chung gồm PostgreSQL, Apache Kafka/ZooKeeper, Elasticsearch, Keycloak, Redis, cert-manager và NGINX Ingress Controller. Các thành phần này cung cấp lưu trữ dữ liệu, xác thực, messaging, tìm kiếm, chứng chỉ và điểm vào HTTP cho các service của YAS.
- Phạm vi ứng dụng triển khai gồm 14 service chính: product, cart, order, customer, inventory, tax, media, search, storefront-bff, storefront-ui, backoffice-bff, backoffice-ui, swagger-ui và sampledata; đảm bảo bao phủ luồng thương mại điện tử phía người dùng, luồng quản trị, dữ liệu mẫu, API documentation và các phụ thuộc cần thiết cho demo Service Mesh.
- Về triển khai, nhóm đóng gói YAS bằng Helm chart theo từng service và một Helm umbrella chart k8s/charts/yas-umbrella. Hai môi trường dev và staging được mô tả bằng các file values riêng trong k8s/environments và được ArgoCD đồng bộ theo mô hình GitOps. Phần Service Mesh sử dụng Istio và Kiali để cấu hình mTLS, AuthorizationPolicy và retry policy.

```
DOKS cluster - nodes & LoadBalancer

$ kubectl get nodes -o wide # DigitalOcean Kubernetes (DOKS) - region sgp1, k8s v1.36.0
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP
devops-project02-mem-opt-3c02m0 Ready <none> 32h v1.36.0 10.104.0.6 178.128.19.54
devops-project02-mem-opt-3c02m1 Ready <none> 32h v1.36.0 10.104.0.5 139.59.96.233
devops-project02-mem-opt-3c02m9 Ready <none> 32h v1.36.0 10.104.0.8 168.144.110.107
devops-project02-mem-opt-3c02mz Ready <none> 32h v1.36.0 10.104.0.2 165.22.100.49

$ kubectl get svc -n ingress-nginx ingress-nginx-controller
NAME TYPE EXTERNAL-IP PORT(S)
ingress-nginx-controller LoadBalancer 129.212.208.194 80:31107/TCP,443:30917/TCP
```

Hình 1: Cụm DOKS: 4 worker node (v1.36.0, region sgp1) và NGINX Ingress Controller kiểu LoadBalancer với IP công khai 129.212.208.194.

**k8s-1-36-0-do-1-sgp1-1782356447895**

in  first-project / SGP1 - 1.36.0-do.1

Actions

OverviewResourcesInsightsMarketplaceSettings

Overview

Learn

CONFIGURATION

CPU

16 vCPUs

Memory

32 GiB

Disk

640 GiB

Download Config File

Copy Config File

Remind me how to use this file to connect to the cluster

NETWORKING

VPC network

 default-sgp1

Node network

10.x.x.x/x

Pod network

10.x.x.x/x

Service network

10.x.x.x/x

NODE POOL STATUS

4/4 Running

devops-project02-mem-opt

4/4 nodes running

View resource details

CONTROL PLANE

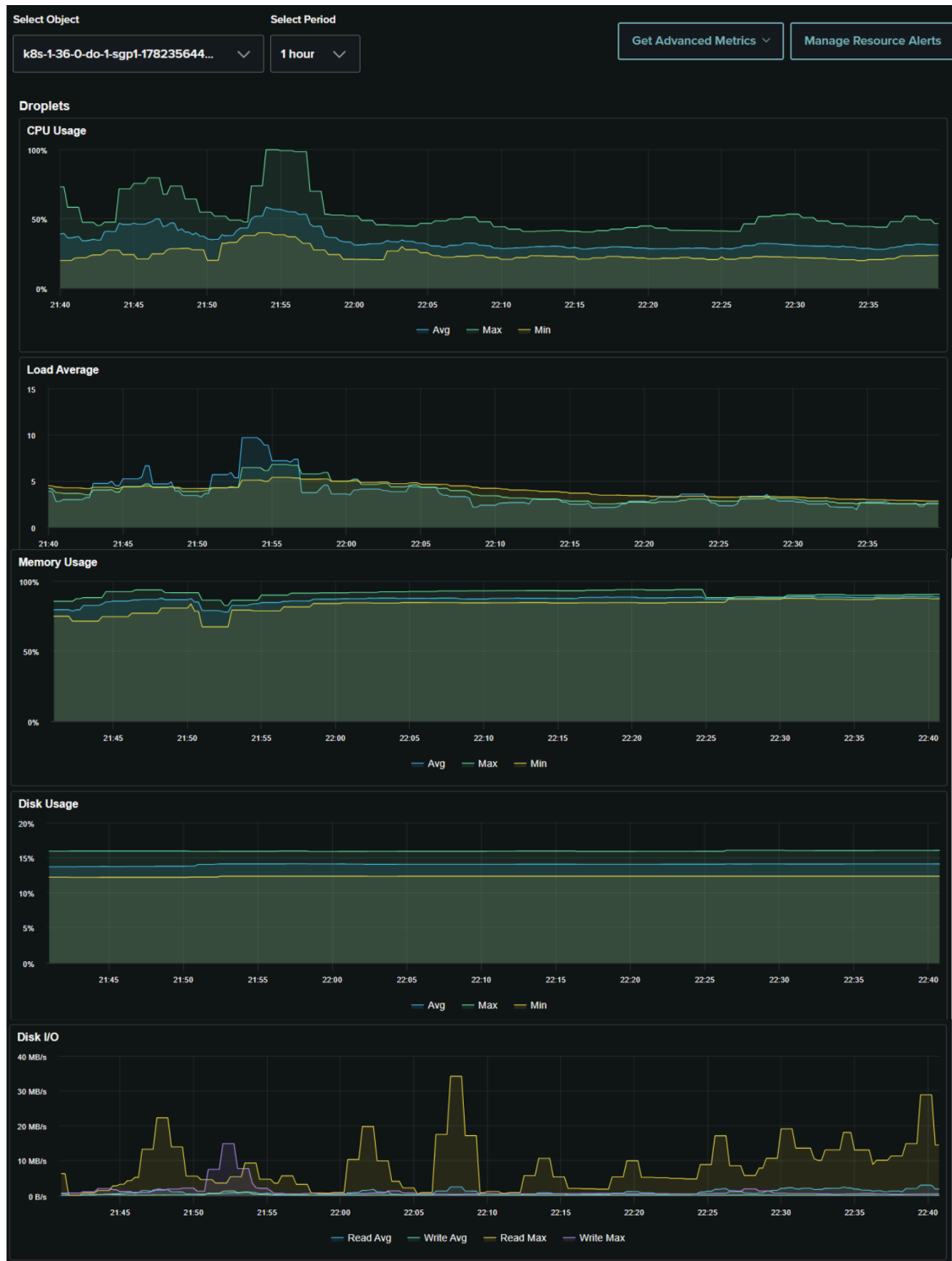
High availability

Not enabled

A high availability control plane creates multiple replicas of control plane components to maximize cluster access and uptime with a 99.95% SLA for \$40/month.

Add High Availability

Hình 2: Giao diện tổng quan của Kubernetes Cluster trên DigitalOcean



Hình 3: Giao diện giám sát tài nguyên của Kubernetes Cluster trên DigitalOcean

2.2. CI: Build image theo commit và push lên registry

Đối với các nhánh phát triển (không phải main), workflow CI được kích hoạt khi có push. Workflow sử dụng matrix build cho 14 service. Với các service backend, pipeline thiết lập JDK 25, build jar bằng Maven với lệnh package và bỏ qua test để rút ngắn thời gian tạo artifact; sau đó Docker Buildx build image và push lên GHCR. Mỗi image được gắn tag bằng github.sha, tức commit SHA của lần commit kích hoạt workflow, đồng thời có thêm tag theo tên branch đã được chuẩn hóa.

```
on:
  push:
    branches-ignore: ["main"]
    tags-ignore: ["v*"]
  ...
image="ghcr.io/23122004/${{ matrix.image }}"
tags="${{image}}:${{ github.sha }}"           # tag = commit id
safe_branch="$(echo "${{ github.ref_name }}" | tr '/' '-' ...)"
tags="${{tags}},${{image}}:${{safe_branch}}"   # tag = tên branch
if [ "${{ github.ref_name }}" = "main" ]; then
  tags="${{tags}},${{image}}:latest"           # main -> latest
fi
```

Cách gắn tag này đáp ứng yêu cầu khi developer commit lên một branch, hệ thống sinh ra image có tag là commit id cuối cùng của branch đó. Tag commit SHA là nguồn tham chiếu chính để truy vết từ mã nguồn sang image và lần triển khai; tag branch chỉ đóng vai trò nhãn tiện lợi để tra cứu trên registry. Riêng luồng main/dev và release/staging được tách sang các workflow CD tương ứng: deploy-dev.yml build image cho main với tag commit SHA và main, còn deploy-staging.yml build image theo Git tag dạng v*.

Platform Solutions Resources Open Source Enterprise Pricing

Search or jump to...

Sign in Sign up

23122004 / Y3S2-DevOps Public

forked from nashtech-garage/yas

Code Pull requests Actions Projects Security and quality Insights

yas-product hotfix-charts-ci Public Latest

Installation OS / Arch Learn more about packages

Install from the command line

```
$ docker pull ghcr.io/23122004/yas-product:hotfix-charts-ci
```

Recent tagged image versions

hotfix-charts-ci	ae4a1cb2438aff31674f93fea1623a046fa4a439	2
Published about 2 hours ago · Digest		
c437554acde39f9991a205352dbac989ee611ef	main	5
Published about 16 hours ago · Digest		
c1f042314ca40543244b74347b2b414fe8049447		4
Published about 23 hours ago · Digest		
642495ea82b0937130e49eed933ce7cb7365c14b		5
Published 1 day ago · Digest		
c7a1d9e9c71dbcd15aab3c2c72a8a2f1768b66		3
Published 1 day ago · Digest		

View all tagged versions

README.md

YAS: Yet Another Shop

Details

23122004

Y3S2-DevOps

MIT License

1 star

Last published

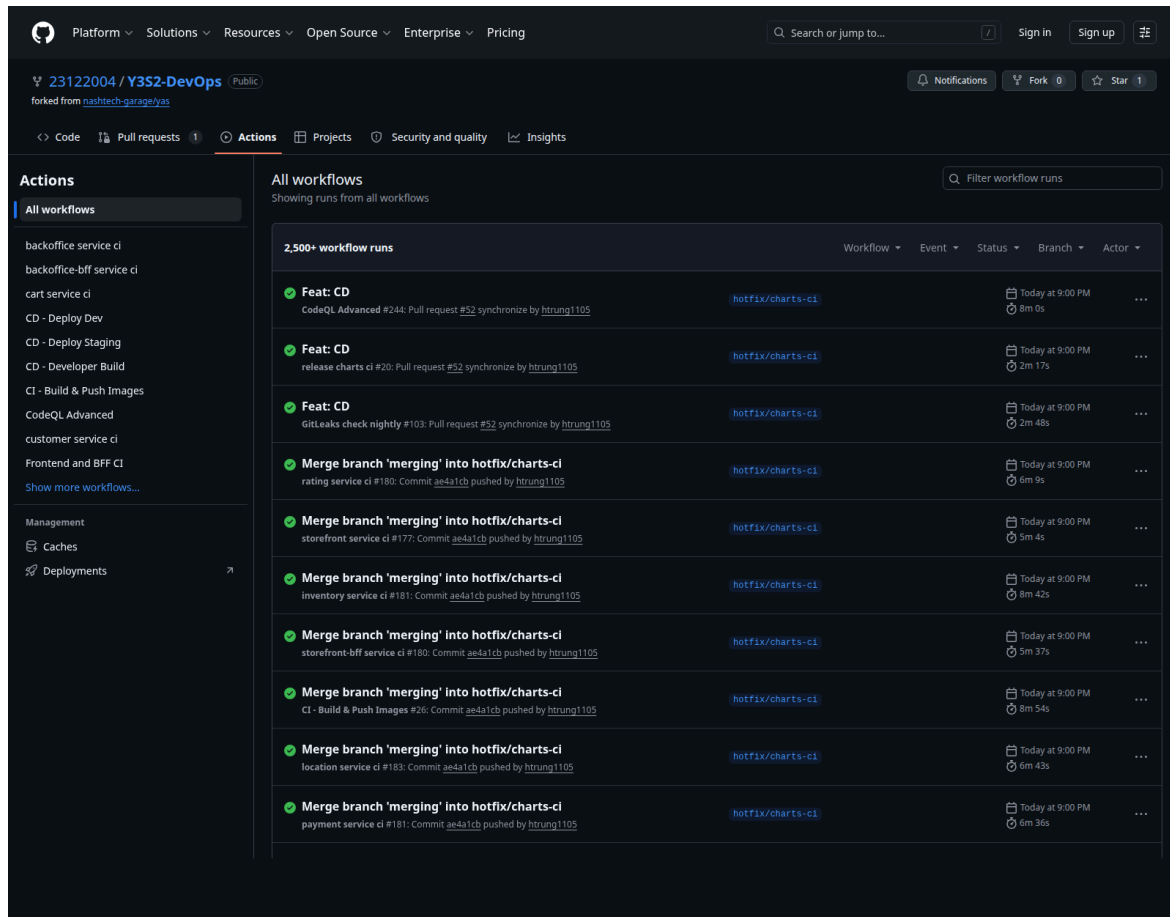
2 hours ago

Total downloads

126

Contributors

Hình 4: GHCR package yas-product: mỗi commit tạo một image gắn tag commit-SHA (vd. c437554acde...) kèm tag branch/main; ảnh public, forked từ nashtech-garage/yas.



Hình 5: Tab Actions của repository: các workflow CI theo service và các workflow CD (Deploy Dev, Deploy Staging, Developer Build) đều chạy thành công.

2.3. Xây dựng cluster K8S và triển khai YAS

Cụm Kubernetes được khởi tạo trên DigitalOcean Kubernetes. Sau khi có cluster, nhóm cấu hình kubeconfig bằng doctl và cài các thành phần nền phục vụ triển khai: NGINX Ingress Controller làm gateway HTTP, cert-manager cho quản lý chứng chỉ, ArgoCD cho GitOps, Istio/Kiali cho Service Mesh, cùng các dịch vụ hạ tầng như PostgreSQL, Kafka/ZooKeeper, Elasticsearch, Keycloak và Redis. Cách tổ chức này tách rõ hạ tầng dùng chung với các service nghiệp vụ của YAS, giúp việc triển khai và kiểm tra từng lớp hệ thống dễ theo dõi hơn.

Ứng dụng YAS được đóng gói bằng hệ thống Helm chart trong thư mục k8s/charts. Các chart theo từng service được gom bởi Helm umbrella chart

k8s/charts/yas-umbrella; chart yas-configuration cung cấp ConfigMap/Secret dùng chung để các service đọc cấu hình kết nối hạ tầng. Các namespace chính gồm dev, staging, yas và yas-developer đều được khai báo trong k8s/namespaces.yaml, đồng thời bật nhãn istio-injection=enabled để sẵn sàng tự động thêm sidecar khi chạy trong mesh. Việc deploy dev/staging được ArgoCD đồng bộ từ Git, còn môi trường developer được deploy thủ công qua workflow ở mục 2.4.

```
D:\Y3S2\[DevOps]_Project02>kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
backoffice-bff-5868f46d99-2qj95	2/2	Running	0	64m
backoffice-ui-56fbc4cdd7-dzqhl	2/2	Running	0	64m
cart-6b559db5c4-j7s1x	2/2	Running	0	64m
customer-945999d65-tfshq	2/2	Running	0	64m
dev-yas-app-swagger-ui-7cbcf5bdf6-4dth9	2/2	Running	1 (62m ago)	64m
inventory-cb48cb585-w6njt	2/2	Running	0	64m
media-666bbf8c75-jtrt6	2/2	Running	0	64m
order-5f76b4c654-87k56	2/2	Running	0	63m
product-786cf48776-tlnhp	2/2	Running	0	64m
sampledata-c6df8fb86-xhpz5	2/2	Running	0	64m
search-84f575675f-vq4s5	2/2	Running	0	64m
storefront-bff-b8c8fb4c7-r2xb5	2/2	Running	0	64m
storefront-ui-65b4dc57c4-lprlj	2/2	Running	0	64m
tax-5cdc67664c-9c5wv	2/2	Running	0	64m
yas-reloader-5c67d449c7-4b5nr	2/2	Running	0	47h

Hình 6: 14 service của YAS ở trạng thái Running trong namespace dev; cột READY 2/2 cho thấy sidecar istio-proxy đã được tự động thêm vào mỗi pod.

```
D:\Y3S2\[DevOps]_Project02>kubectl get svc -n ingress-nginx
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
ingress-nginx-controller	LoadBalancer	10.109.21.51	129.212.208.194	80:31107/TCP,443:30917/TCP	14d
ingress-nginx-controller-admission	ClusterIP	10.109.12.28	<none>	443/TCP	14d
ingress-nginx-controller-metrics	ClusterIP	10.109.10.130	<none>	9090/TCP	14d

Hình 7: ingress-nginx LoadBalancer, host routing cho storefront/backoffice/api.

Sau khi Helm/ArgoCD triển khai ứng dụng, phần kiểm chứng không chỉ dừng ở trạng thái pod Running mà cần chứng minh từng nhóm service phục vụ đúng vai trò trong hệ thống YAS. Nhóm sử dụng ba lớp bằng chứng: trạng thái Kubernetes, health/API endpoint và giao diện người dùng hoặc quản trị.

Free shipping for standard order over \$100

[Notification](#)
[Help & FAQs](#)
[EN](#)
[Login](#)

Yas - Storefront

[Search](#)

[Phone](#)
[Laptop](#)
[iPhone](#)
[Workstation](#)
[Galaxy](#)
[iPad](#)
[Tablet](#)
[Macbook](#)

CATEGORIES

Phone

Galaxy

Laptop

iPad

Macbook

Yas

[Catalog](#)
[Brands](#)
[Categories](#)
[Products](#)
[Product Options](#)
[Product Attributes](#)
[Product Attribute Groups](#)
[Product Templates](#)
[Customers](#)
[Sales](#)
[Location](#)
[Inventory](#)
[Promotion](#)

Signed in as: admin Logout

List of the 5 latest Products

ID	Name	Slug	Created On	Action
17	iPad 10.9-Inch (10th Generation)	ipad-109-inch-10th-generation		Details
16	Dell Inspiron 16 5640 Laptop 16"	dell-inspiron-16-5640-laptop-16		Details
15	Samsung Galaxy Z Flip6	samsung-galaxy-z-flip6		Details
14	MacBook Air M3	macbook-air-m3		Details
13	MacBook Air M2	macbook-air-m2		Details

List of the 5 latest Orders

ID	Email	Status	Created On	Action
No Orders available				

List of the 5 latest Ratings

ID	Product Name	Content	Created On	Action
No Ratings available				

Swagger

Select a definition Product

Product Service API 1.0 OAS 3.1

<http://api.dev.yas.local.com/product/v3/api-docs>
 Product API documentation

Servers

/product - Default Server URL

Authorize

product-controller

PUT /backoffice/products/{id}

DELETE /backoffice/products/{id}

PUT /backoffice/products/update-quantity

PUT /backoffice/products/subtract-quantity

GET /backoffice/products

Hình 8: Storefront, Backoffice và Swagger UI truy cập được sau khi deploy.

2.4. Job developer_build và truy cập theo domain

Yêu cầu developer_build được thực hiện bằng workflow cd-developer.yml, chạy thủ công qua workflow_dispatch. Workflow cho phép chọn action deploy hoặc cleanup, chọn developer_profile lean/full, và nhập branch riêng cho từng service như product, cart, order, tax, storefront-bff, storefront-ui, backoffice-bff, backoffice-ui, sampledata và swagger-ui. Khi action=deploy, workflow kết nối DOKS bằng doctl, chuẩn bị Helm, tạo/cập nhật namespace yas-developer, cài chart yas-configuration, rồi helm upgrade --install từng service với image tag tương ứng.

```
inputs:
  action: [deploy | cleanup]
  developer_profile: [lean | full]
  product_branch, cart_branch, order_branch, tax_branch, ...
  (default: main)
...
resolve_tag() {
  if [ "$branch" = "main" ]; then echo "latest"; return; fi
  git ls-remote --heads origin "$branch" | awk '{print $1}' #
  commit SHA
}
helm upgrade --install "$release" ./k8s/charts/$release -n
yas-developer \
  --set backend.image.tag="$tag"
```

Cơ chế chọn image của workflow bám sát yêu cầu đề bài: service nào được chỉ định branch phát triển sẽ được phân giải commit SHA mới nhất bằng git ls-remote và deploy image có tag chính là SHA đó; các service không được chọn giữ giá trị mặc định. Theo thiết kế hiện tại, branch main được ánh xạ về tag latest, còn các branch khác dùng commit SHA. Sau khi deploy, workflow lấy địa chỉ LoadBalancer của ingress-nginx và ghi các URL/hosts entries vào GitHub Step

Summary. Ví dụ khi chạy với `action=deploy`, `developer_profile=full`, `product_branch=feature/product-filter`, `storefront_ui_branch=feature/homepage-ui` và các service còn lại dùng `main`, workflow sẽ deploy các image tương ứng vào namespace `yas-developer`. Với `BASE_DOMAIN=yas.local.com` và LoadBalancer IP của `ingress-nginx` là `129.212.208.194`, các host `developer.yas.local.com`, `backoffice-developer.yas.local.com` và `api-developer.yas.local.com` được ghi vào Step Summary để developer thêm vào file hosts và kiểm thử trực tiếp.

Nhóm sử dụng Ingress qua LoadBalancer thay cho NodePort để phù hợp với DOKS managed cluster, đồng thời vẫn cung cấp domain/host cho developer tự trở trong file hosts và kiểm thử trực tiếp.

```
### Developer environment (GitHub Step Summary)
- Storefront: http://developer.${BASE_DOMAIN}
- Backoffice: http://backoffice-developer.${BASE_DOMAIN}
- Swagger UI:
http://api-developer.${BASE_DOMAIN}/swagger-ui/index.html
Hosts entries:
<LB_IP> developer.${BASE_DOMAIN}
<LB_IP> backoffice-developer.${BASE_DOMAIN}
<LB_IP> api-developer.${BASE_DOMAIN}
```

Event ▾ Status ▾ Branch ▾ Actor ▾

Run workflow ▾

Use workflow from

Branch: main ▾

Deploy or cleanup the developer environment *

deploy ▾

Runtime profile for developer environment *

full ▾

Branch for product

main

Branch for cart

main

Branch for order

main

Branch for customer

main

Run workflow

Hình 9: Form chạy thử công workflow CD - Developer Build trên GitHub Actions, chọn **action=deploy**, **developer_profile=full** và **branch main** cho các service trước khi triển khai vào namespace **yas-developer**

2.5. Job xóa triển khai

Nhóm hiện thực job xóa môi trường developer ngay trong workflow **CD - Developer Build** bằng input **action=cleanup**. Khi developer chọn cleanup, workflow kết nối tới DOKS bằng **doctl**, lấy kubeconfig, sau đó xóa namespace **yas-developer**. Toàn bộ tài nguyên của môi trường developer

được deploy vào namespace riêng **yas-developer**, nên xoá namespace sẽ xoá đồng thời pods, services, ingress, Helm resources và config liên quan. Cách này không ảnh hưởng đến namespace **dev** và **staging**.

```
cleanup:
  if: ${ inputs.action == 'cleanup' }
  steps:
    - name: Delete developer deployment
      run: |
        kubectl delete namespace "$NAMESPACE" --ignore-not-found=true
```

2.6. Nâng cao: Quản lý dev/staging bằng ArgoCD – GitOps

Thay vì tạo trực tiếp hai job Jenkins để deploy **dev** và **staging**, nhóm sử dụng ArgoCD theo mô hình GitOps. Trong mô hình này, Git là nguồn sự thật duy nhất. Mọi thay đổi về image tag, cấu hình môi trường và Helm values đều được commit vào repository; ArgoCD theo dõi repository và tự đồng bộ cluster về đúng trạng thái khai báo.

Hai ArgoCD Application chính:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dev-yas-app
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/23122004/Y3S2-DevOps.git
    targetRevision: main
    path: k8s/charts/yas-umbrella
  helm:
```

```
valueFiles:
  - ../../environments/dev/values.yaml
destination:
  server: https://kubernetes.default.svc
  namespace: dev
syncPolicy:
  automated:
    prune: true
    selfHeal: true
syncOptions:
  - CreateNamespace=true
```

Với môi trường **dev**, khi có thay đổi trên branch **main**, workflow **deploy-dev.yml** build image cho các service, push lên GHCR, sau đó cập nhật image tag trong **k8s/environments/dev/values.yaml** bằng commit SHA:

```
yq -i '.product.backend.image.tag = "${{ github.sha }}"'
k8s/environments/dev/values.yaml
yq -i '.cart.backend.image.tag = "${{ github.sha }}"'
k8s/environments/dev/values.yaml
yq -i '.order.backend.image.tag = "${{ github.sha }}"'
k8s/environments/dev/values.yaml
```

vào repository; ArgoCD theo dõi repository và tự đồng bộ cluster về đúng trạng thái khai báo.

Sau khi file values được commit lại, ArgoCD phát hiện thay đổi và tự động sync vào namespace **dev**. Cơ chế này đáp ứng yêu cầu “main thay đổi thì tự động deploy đè vào dev”.

Với môi trường **staging**, workflow **deploy-staging.yml** chạy khi có Git tag dạng **v***, ví dụ **v0.1.0**. Image được build với tag release đó, sau đó cập nhật vào **k8s/environments/staging/values.yml**. Nhờ vậy staging luôn gắn với một version rõ ràng, phù hợp hơn cho kiểm thử release.

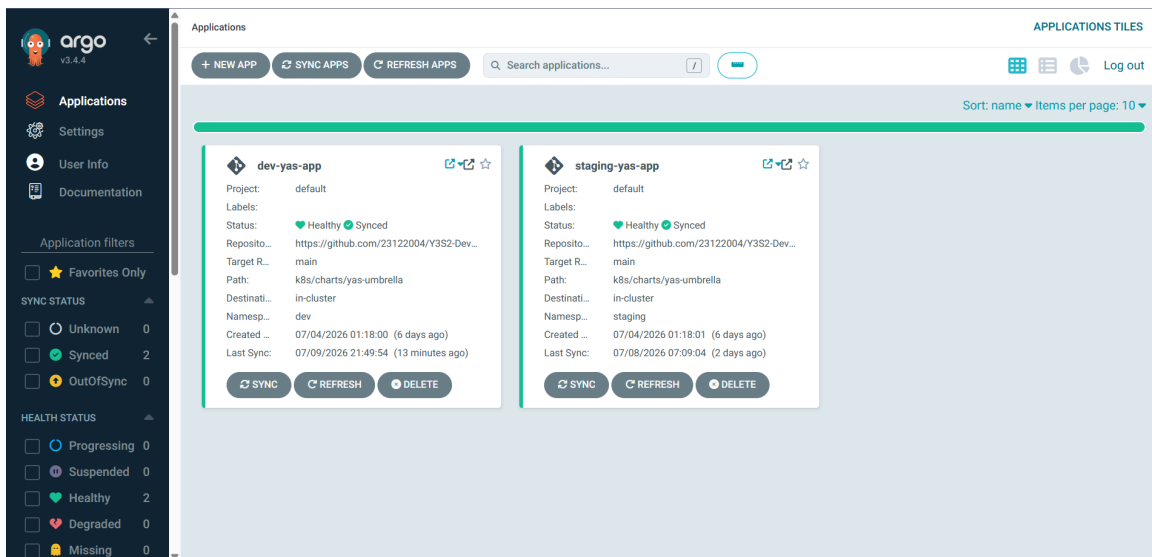
```
ArgoCD Applications + Ingress hosts

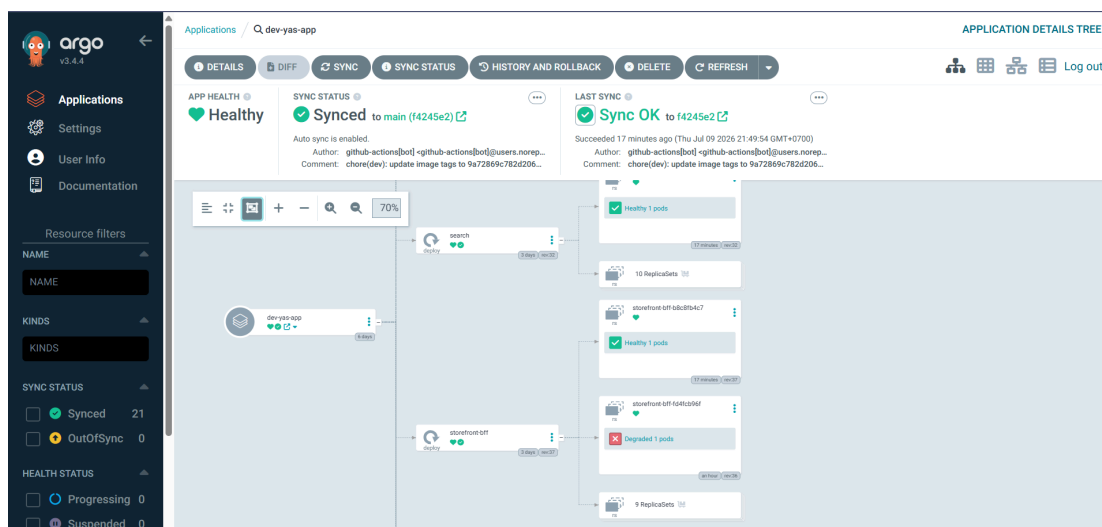
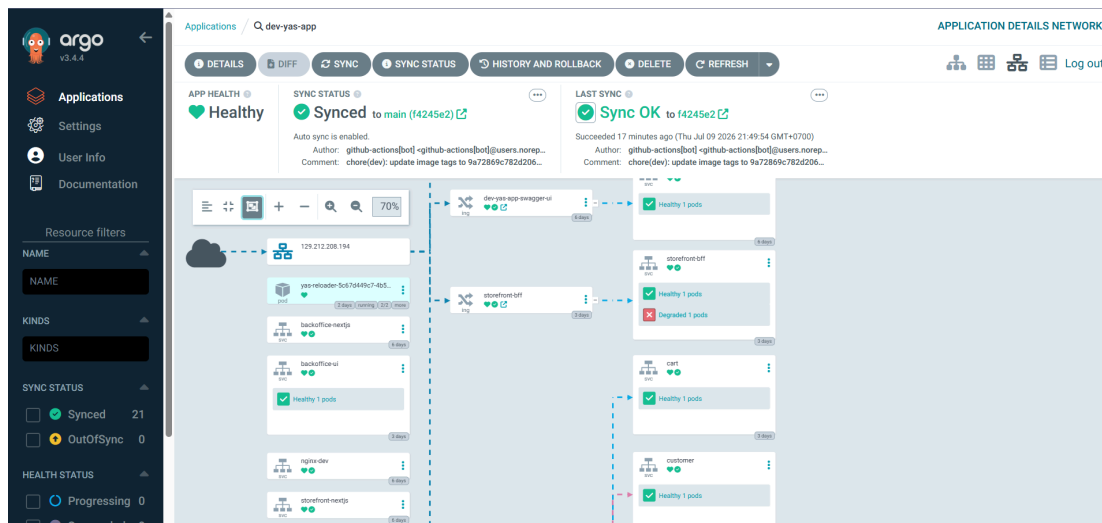
$ kubectl get applications -n argocd
NAME          SYNC STATUS  HEALTH STATUS
dev-yas-app    Synced       Healthy
staging-yas-app Synced       Healthy

$ kubectl get ingress -n dev
NAME          HOSTS                                ADDRESS
storefront-bff storefront.dev.yas.local.com 129.212.208.194
backoffice-bff backoffice.dev.yas.local.com 129.212.208.194
dev-yas-app-swagger-ui api.dev.yas.local.com 129.212.208.194

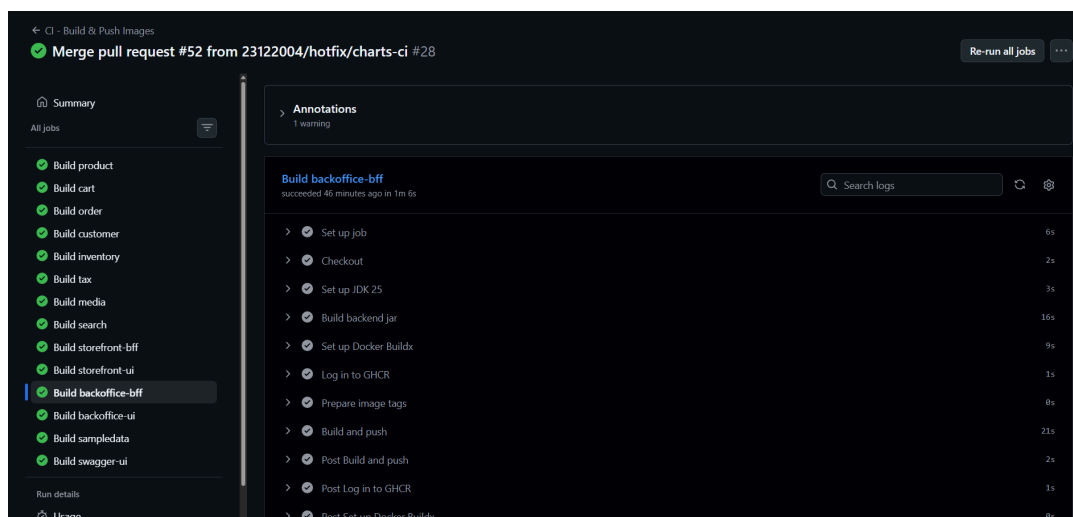
$ kubectl get ingress -n staging
NAME          HOSTS                                ADDRESS
storefront-bff storefront.staging.yas.local.com 129.212.208.194
backoffice-bff backoffice.staging.yas.local.com 129.212.208.194
staging-yas-app-swagger-ui api.staging.yas.local.com 129.212.208.194
```

Hình 10: Hai ArgoCD Application *dev-yas-app* và *staging-yas-app* đều ở trạng thái *Synced/Healthy*; Ingress định tuyến theo host cho từng môi trường.





Hình 11: Screenshot ArgoCD với GitOps quản lý tự động dev/staging



← CD - Deploy Dev

fix: CI #26

Summary

All jobs

Build product

Build cart

Build order

Build customer

Build inventory

Build tax

Build media

Build search

Build storefront-bff

Build storefront-ui

Build backoffice-bff

Build backoffice-ui

Build sampledata

Build swagger-ui

Update GitOps Config

Triggered via push 50 minutes ago

htrung1105 pushed · f2a25fe · main

Status

Success

Total duration

5m 21s

Artifacts

14

deploy-dev.yml

on: push

Matrix build-and-push

14 jobs completed

Show all jobs

Update GitOps Config 6s

Build product summary

Docker Build summary

For a detailed look at the build, download the following build record archive and import it into Docker Desktop' Build records include details such as timing, dependencies, results, logs, traces, and other information about a b

← CD - Deploy Staging

update: GHCR #1

Summary

All jobs

Build product

Build cart

Build order

Build customer

Build inventory

Build tax

Build media

Build search

Build storefront-bff

Build storefront-ui

Build backoffice-bff

Build backoffice-ui

Build sampledata

Build swagger-ui

Update GitOps Config

Triggered via push 3 days ago

htrung1105 pushed · f2a25fe · v1.0.0

Status

Success

Total duration

1m 24s

Artifacts

14

deploy-staging.yml

on: push

Matrix build-and-push

14 jobs completed

Show all jobs

Update GitOps Config 9s

Build product summary

Docker Build summary

For a detailed look at the build, download the following build record archive and import it into Docker Desktop' Build records include details such as timing, dependencies, results, logs, traces, and other information about a b

Hình 12: Screenshot GitHub Actions CD - Deploy Dev và CD - Deploy Staging chạy thành công.

2.7. Nâng cao: Service Mesh với Istio và Kiali

Nhóm triển khai Service Mesh bằng **Istio** để bổ sung lớp kiểm soát giao tiếp giữa các microservice mà không cần sửa mã nguồn ứng dụng. Các cấu hình chính được đặt trong thư mục **k8s/istio/**, bao gồm mTLS, chính sách phân quyền service-to-service, retry policy và Kiali để quan sát topology.

Trong phạm vi kiểm thử Service Mesh, nhóm dùng namespace **yas** làm môi trường mesh chính. Namespace này được cấu hình mTLS ở chế độ **STRICT**, nghĩa là các kết nối service-to-service phải đi qua Envoy sidecar và được xác thực bằng chứng chỉ do Istio cấp. Với các namespace phục vụ demo giao diện như **dev** và **staging**, cấu hình mTLS được để **PERMISSIVE** nhằm tương thích với NGINX Ingress và các endpoint public như **storefront-bff**, **backoffice-bff**, **swagger-ui**.

Đoạn cấu hình mTLS rút gọn:

```
kind: PeerAuthentication
metadata:
  name: default-strict-mtls
  namespace: yas
spec:
  mtls:
    mode: STRICT
```

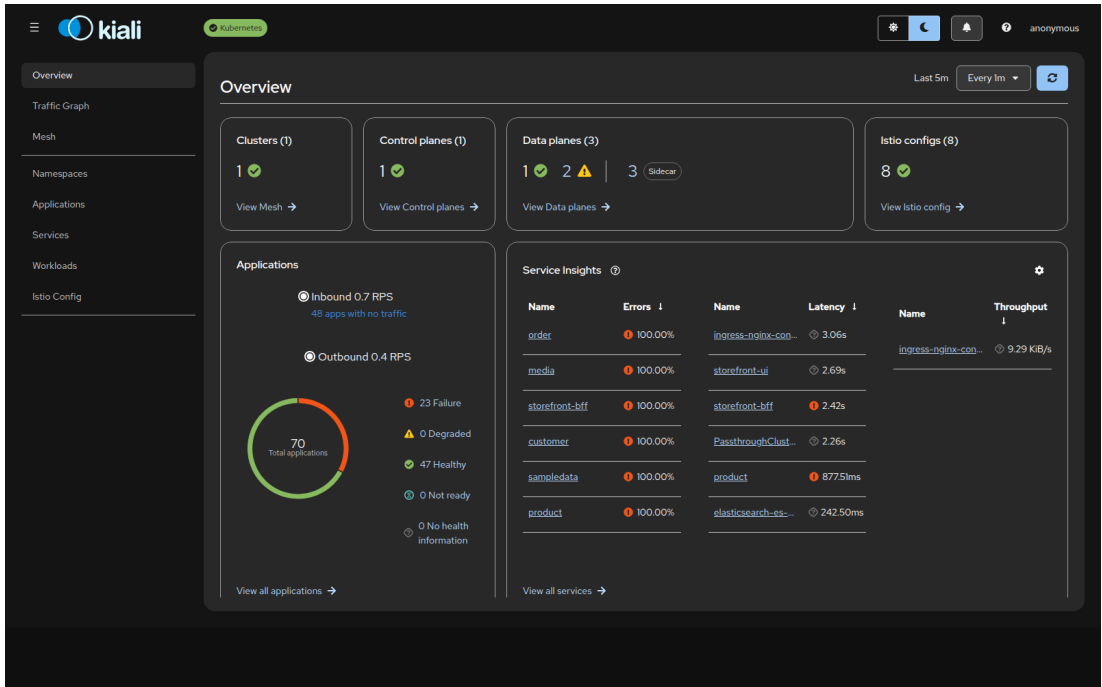
Các pod ứng dụng được thêm sidecar của Istio. Vì vậy, ngoài container ứng dụng, pod còn có thành phần **istio-proxy** để chặn, mã hoá, định tuyến và ghi nhận traffic. Đây là nền tảng để Istio thực thi mTLS, AuthorizationPolicy và retry mà không cần thay đổi code của service.

```
Istio - mTLS PeerAuthentication & sidecar

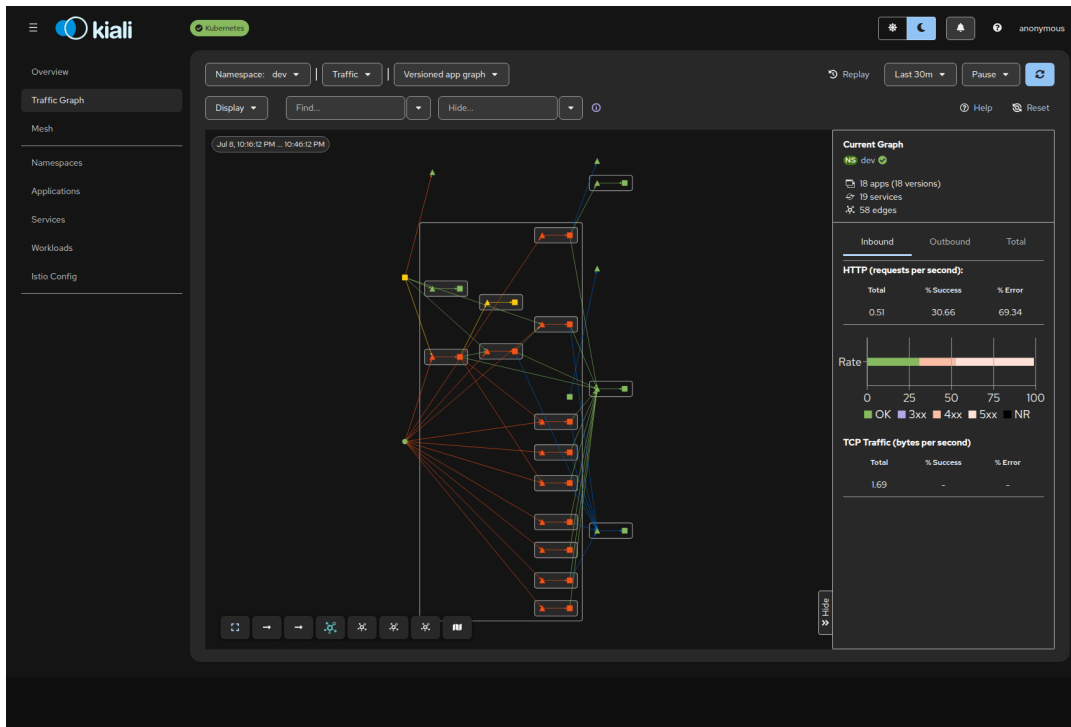
$ kubectl get peerauthentication -A
NAMESPACE   NAME                                     MODE
dev          default-strict-mtls                     PERMISSIVE
dev          storefront-bff-public-entripoint        PERMISSIVE
staging      default-strict-mtls                     PERMISSIVE
staging      storefront-bff-public-entripoint        PERMISSIVE
# STRICT mTLS is defined for the in-mesh namespaces (yas / yas-developer);
# public BFF/Swagger endpoints stay PERMISSIVE for NGINX Ingress.

$ kubectl get pod -n dev product-774dfcb5b5-hmtz6 \
-o jsonpath='{.spec.initContainers[*].name}'
istio-init istio-proxy opentelemetry-auto-instrumentation-java
```

Hình 13: Kiểm tra PeerAuthentication và sidecar Istio: **dev** và **staging** đang ở chế độ PERMISSIVE để tương thích với NGINX Ingress, trong khi STRICT mTLS được dùng cho namespace mesh chính **yas/yas-developer**; pod ứng dụng có các thành phần Istio như **istio-init** và **istio-proxy**.



Hình 14: Kiali Overview: 1 control plane (istiod), 3 data plane có sidecar, 8 Istio config; tổng quan sức khỏe các application trong mesh.



Hình 15: Kiali Traffic Graph của namespace dev: topology và lưu lượng thực đi qua sidecar Envoy giữa các service (18 app, 19 service, 58 cạnh).

Sau khi cài Istio, nhóm cài thêm **Kiali** để quan sát trạng thái Service Mesh. Kiali cho biết control plane **istiod**, data plane và các cấu hình Istio đang được nhận diện trong cluster. Ảnh overview cho thấy Kiali đã kết nối được tới Kubernetes cluster, nhận diện control plane, data plane có sidecar và các Istio config đang áp dụng.

Kiali Traffic Graph được dùng để quan sát luồng gọi giữa các service. Trong ảnh, namespace đang quan sát là **dev**, hiển thị **18 apps**, **19 services** và **58 edges**. Điều này chứng minh các service YAS đã phát sinh traffic thực tế và Kiali có thể dựng topology từ metric của Envoy sidecar.

Để giới hạn service nào được phép gọi service nào, nhóm dùng **AuthorizationPolicy**. Ví dụ với **product-service**, chỉ các caller hợp lệ trong luồng nghiệp vụ như **order**, **cart**, **search**, **storefront-bff** và

backoffice-bff được phép truy cập. Các workload identity khác không nằm trong allow-list sẽ bị Envoy chặn với HTTP 403.

Đoạn policy rút gọn:

```
kind: AuthorizationPolicy
metadata:
  name: product-allow-callers
  namespace: yas
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: product
  action: ALLOW
```

Retry policy được cấu hình bằng **VirtualService**. Với các service như **tax** và **order**, khi upstream trả lỗi **5xx**, Envoy sẽ tự retry tối đa 3 lần. Cơ chế này nằm ở tầng Service Mesh, nên không cần sửa code của từng service.

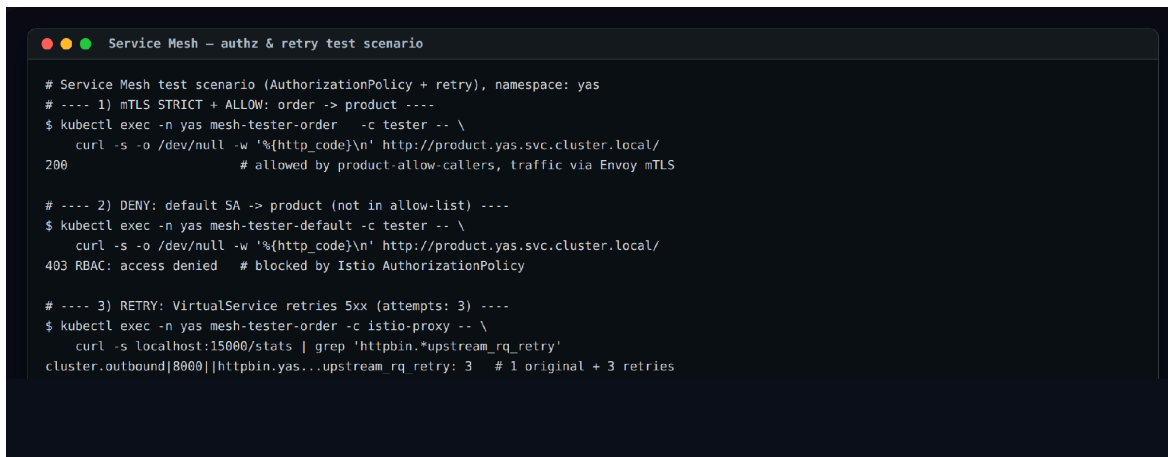
Nhóm chuẩn bị kịch bản test trong namespace **yas** với hai pod client:

- **mesh-tester-order**: dùng ServiceAccount **order**, nằm trong allow-list nên được phép gọi **product**.
- **mesh-tester-default**: dùng ServiceAccount **default**, không nằm trong allow-list nên bị chặn.

Kết quả test:

- **order -> product** trả HTTP **200**, chứng minh request được cho phép và đi qua Envoy/mTLS.
- **default -> product** trả HTTP **403 RBAC: access denied**, chứng minh AuthorizationPolicy đã chặn đúng.

- Retry policy với `httpbin /status/500` ghi nhận `upstream_rq_retry: 3`, nghĩa là một request lỗi sinh ra 3 lần retry ở Envoy.



```
# Service Mesh test scenario (AuthorizationPolicy + retry), namespace: yas
# ---- 1) mTLS STRICT + ALLOW: order -> product ----
$ kubectl exec -n yas mesh-tester-order -c tester -- \
  curl -s -o /dev/null -w '%{http_code}\n' http://product.yas.svc.cluster.local/
200 # allowed by product-allow-callers, traffic via Envoy mTLS

# ---- 2) DENY: default SA -> product (not in allow-list) ----
$ kubectl exec -n yas mesh-tester-default -c tester -- \
  curl -s -o /dev/null -w '%{http_code}\n' http://product.yas.svc.cluster.local/
403 RBAC: access denied # blocked by Istio AuthorizationPolicy

# ---- 3) RETRY: VirtualService retries 5xx (attempts: 3) ----
$ kubectl exec -n yas mesh-tester-order -c istio-proxy -- \
  curl -s localhost:15000/stats | grep 'httpbin.*upstream_rq_retry'
cluster.outbound[8000]|httpbin.yas...upstream_rq_retry: 3 # 1 original + 3 retries
```

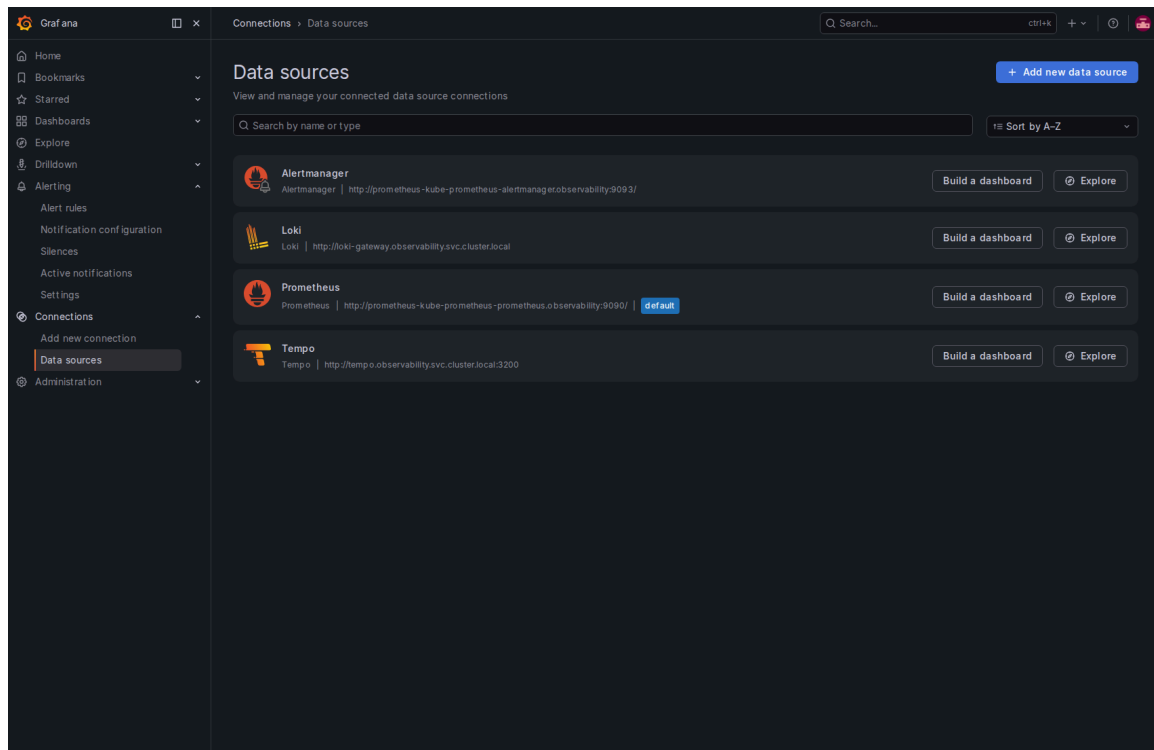
Hình 16: Kịch bản kiểm thử Service Mesh trong namespace `yas`: `order` gọi `product` thành công với HTTP 200 qua Envoy/mTLS; `ServiceAccount default` bị `AuthorizationPolicy` chặn với HTTP 403; `retry policy` ghi nhận 3 lần retry khi `upstream` trả lỗi 5xx.

2.8. Observability: thu thập log tập trung với Loki, Promtail và Grafana

Nhóm bổ sung luồng thu thập log tập trung ngay trên cụm Kubernetes. Promtail thu log từ toàn bộ pod và đẩy vào Loki để lưu trữ; Grafana truy vấn và hiển thị log qua Explore bằng ngôn ngữ LogQL. Toàn bộ stack được cài bằng Helm vào namespace `observability`. Nhờ đó có thể lọc log theo namespace, app, pod, container của từng service YAS.

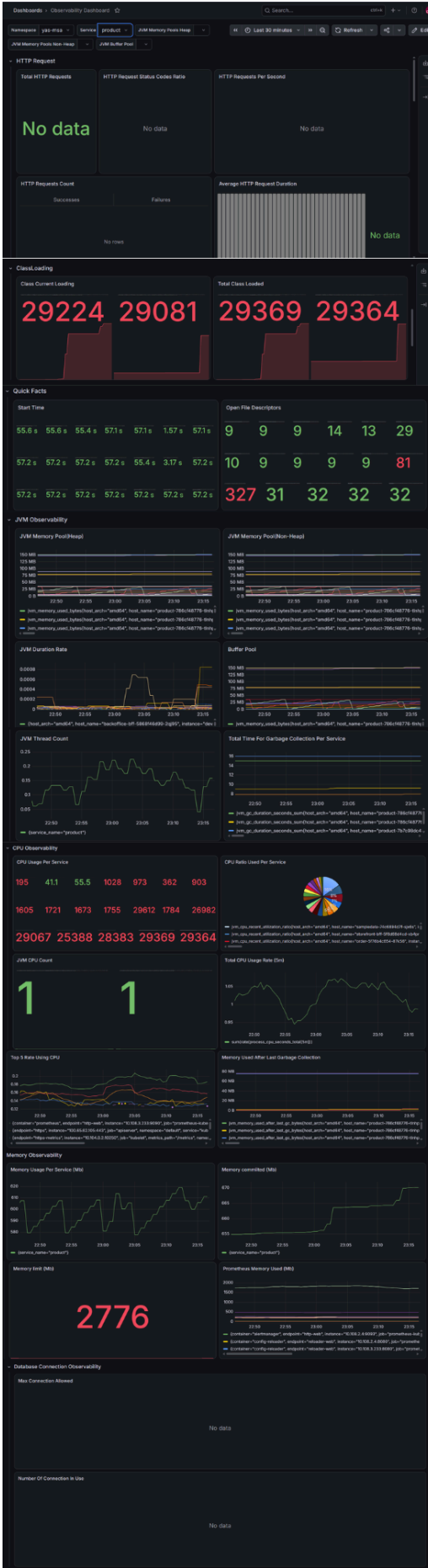
Prometheus được dùng để thu metrics từ các service Spring Boot thông qua endpoint `/actuator/prometheus`. Trong Helm chart backend, nhóm có cấu hình `ServiceMonitor` để Prometheus tự phát hiện và scrape metrics của các service.

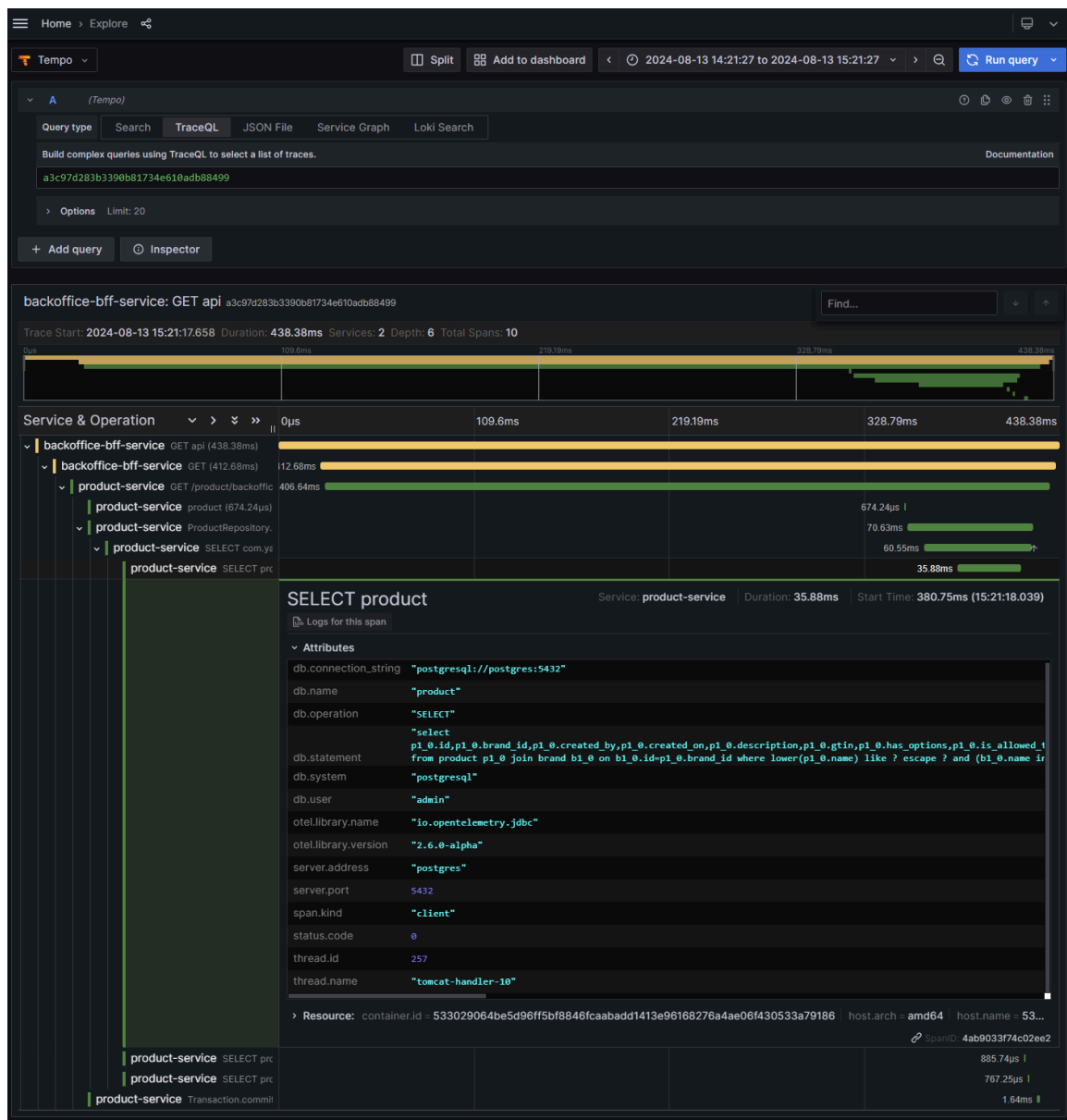
Grafana được dùng làm giao diện quan sát tập trung. Dashboard hiện tại hiển thị các chỉ số quan trọng như request rate, số request thành công/thất bại, JVM memory, JVM CPU, class loading và database connection. Những chỉ số này giúp phát hiện service đang bị lỗi, quá tải hoặc có dấu hiệu bất thường sau mỗi lần deploy.



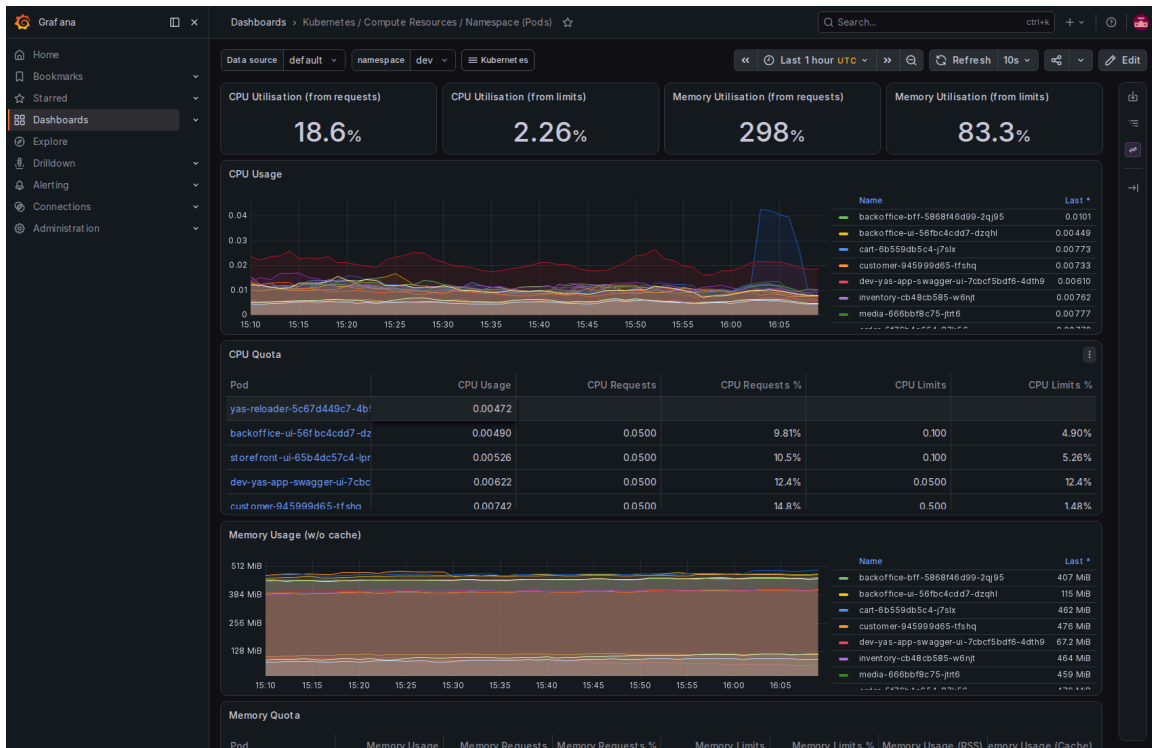
Hình 17: Các nguồn dữ liệu tích hợp bao gồm Alertmanager, Loki (quản lý logs), Prometheus (thu thập metrics hệ thống - mặc định) và Tempo (quản lý traces).

Ngoài metrics, nhóm cấu hình OpenTelemetry để thu distributed tracing. Các trace được gửi về OpenTelemetry Collector rồi lưu trong Tempo. Nhờ đó có thể xem một request đi qua những service nào, mất bao lâu ở từng bước, và truy xuống cả thao tác database như câu lệnh SELECT trong **product-service**.





Hình 18: Giao diện Grafana Explore sử dụng mã nguồn Tempo để truy vết luồng xử lý của yêu cầu **GET api** từ dịch vụ **backoffice-bff-service**. Biểu đồ trực quan hóa chi tiết thời gian phản hồi, các span con và câu lệnh truy vấn **SELECT** cụ thể tới cơ sở dữ liệu PostgreSQL để tối ưu hóa hiệu năng.



Hình 19: Giao diện Grafana hiển thị các thông số sử dụng tài nguyên (CPU, Memory Utilisation/Quota) thực tế của các Pods thuộc namespace dev.

III. Phụ lục

3.1. Thông tin nội bài

- **Repository:** <https://github.com/23122004/Y3S2-DevOps> (fork từ repository nashtech-garage/yas).
- **Registry image:** ghcr.io/23122004/yas-<service> (public).
- **Mã nguồn CI/CD:** .github/workflows/ (deploy-dev.yml, deploy-staging.yml, cd-developer.yml, ci.yml, charts-ci.yaml).
- **Mã nguồn hạ tầng:** k8s/argocd/ (Application), k8s/charts/ (Helm umbrella + subchart), k8s/environments/ (values dev/staging), k8s/istio/ (mesh).

3.2. Quy trình thiết lập (tóm tắt)

1) Kết nối cụm DOKS

```
doctl kubernetes cluster kubeconfig save <DOKS_CLUSTER_NAME>

# 2) Cài thành phần nền
helm install ingress-nginx ...      # Ingress LoadBalancer
kubectl apply -f k8s/istio/         # PeerAuthentication + DestinationRule
                                   # + AuthorizationPolicy + VirtualService
kubectl apply -f k8s/argocd/dev-application.yaml
kubectl apply -f k8s/argocd/staging-application.yaml

# 3) Secrets (Action) trên GitHub: DIGITALOCEAN_ACCESS_TOKEN,
DOKS_CLUSTER_NAME, BASE_DOMAIN

# 4) Vận hành
# - Commit lên branch -> CI build image tag = commit SHA (GHCR)
# - Push main          -> tự deploy vào namespace dev qua ArgoCD
# - Push tag v*        -> deploy release vào namespace staging qua ArgoCD
# - Actions -> CD - Developer Build -> deploy/cleanup môi trường
developer
```

— Hết —